

目录

1 数据一致性模型 2

1.1 顺序一致性 2

1.2 线性一致性 3

1.3 因果一致性 4

1.4 最终一致性 4

2 客户端一致性模型 5

3 副本管理（对应 7.4 Replica management） 6

复制（Replication）

为什么复制？

- 提高可靠性：如果一个副本失效，可以切换到另一个副本。
- 提高性能与可扩展性：分散请求负载，或利用地理位置接近性实现快速响应。

现实问题：

- 拥有多个副本意味着当任何副本发生更改时，该更改必须在所有副本上执行。
- 副本需要保持**一致**。

性能与可扩展性

主要问题：

- 为了保持副本一致性，通常需要确保所有冲突操作在各处以相同顺序执行。
- 冲突操作：来自事务领域的概念。
 - 读-写冲突：读操作和写操作并发执行。
 - 写-写冲突：两个并发写操作。

挑战：保证不发生冲突操作的全局排序可能代价高昂，会降低可扩展性。

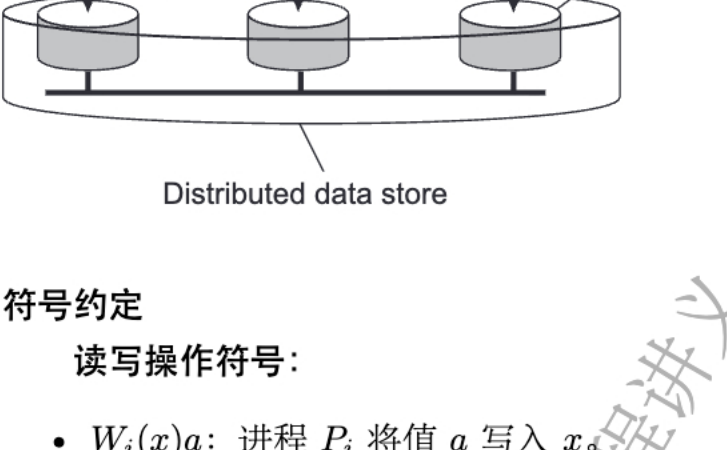
解决方案：弱化一致性要求，以避免全局同步。

1 数据一致性模型

数据一致性模型

一致性模型：

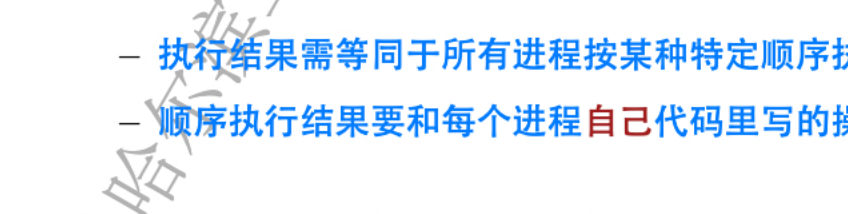
- 分布式数据存储与进程之间的契约。
- 精确说明在并发情况下读写操作的结果。



符号约定

读写操作符号：

- $W_i(x)a$: 进程 P_i 将值 a 写入 x 。
- $R_i(x)b$: 进程 P_i 从 x 读取值 b 。
- 所有数据项初始值为 NIL。



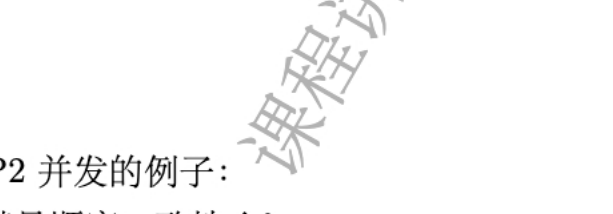
1.1 顺序一致性

顺序一致性

- 顺序一致性 (Sequential Consistency) 定义：**
 - 执行结果需等同于所有进程按某种特定顺序执行，且顺序执行结果要和每个进程自己代码里写的操作顺序一致。
- 例子：**一个共享的在线文档，虽然每个人的更新到达时间可能不同，但
 - 系统最终会保证所有人看到的修改历史是同一个顺序（比如：A 先改，然后 B 改，然后 C 改）
 - 并且 A 的修改一定发生在 B 之前，B 在 C 之前。

2

Execution 1	Execution 2	Execution 3	Execution 4
$P_1: x \leftarrow 1;$ $P_1: \text{print}(y,z);$ $P_2: y \leftarrow 1;$ $P_2: \text{print}(x,z);$ $P_2: z \leftarrow 1;$ $P_2: \text{print}(x,y);$	$P_1: x \leftarrow 1;$ $P_2: y \leftarrow 1;$ $P_1: \text{print}(x,z);$ $P_1: \text{print}(x,z);$ $P_2: z \leftarrow 1;$ $P_1: \text{print}(x,y);$	$P_2: y \leftarrow 1;$ $P_2: z \leftarrow 1;$ $P_1: \text{print}(x,y);$ $P_1: \text{print}(x,z);$ $P_1: \text{print}(x,z);$ $P_2: \text{print}(x,z);$	$P_2: y \leftarrow 1;$ $P_1: x \leftarrow 1;$ $P_2: z \leftarrow 1;$ $P_2: \text{print}(x,z);$ $P_2: \text{print}(x,z);$ $P_1: \text{print}(x,y);$
Prints: 00111 Signature: 0 1 0 1 1	Prints: 10111 Signature: 1 0 1 1 1	Prints: 01111 Signature: 1 1 0 1 1	Prints: 11111 Signature: 1 1 1 1 1



顺序一致性示例

- 三个并发进程（初始值：0）**
 - 进程 P_1 : $x \leftarrow 1$; $\text{print}(y,z)$;
 - 进程 P_2 : $y \leftarrow 1$; $\text{print}(x,z)$;
 - 进程 P_3 : $z \leftarrow 1$; $\text{print}(x,y)$;
- 执行序列：**

小测验

看如下 P_1 与 P_2 并发的例子：
这个运行结果满足顺序一致性么？
不满足，因为我们无法找到一个能产生这样效果的全局序。

1.2 线性一致性

线性一致性

线性一致性 (Linearizability) 定义：

- 每个操作都应看起来在其开始和完成之间的某个时刻瞬间生效。

与顺序一致性的区别：

Possible ordering of operations	Result
$W_1(x)a, W_1(y)a, W_2(y)b, W_2(x)b$	$R_1(x)b, R_2(y)b$
$W_1(x)a, W_2(y)b, W_1(y)a, W_2(x)b$	$R_1(x)b, R_2(y)a$
$W_1(x)a, W_2(y)b, W_2(x)b, W_1(y)a$	$R_1(x)b, R_2(y)a$
$W_2(y)b, W_1(x)a, W_2(x)b, W_1(y)a$	$R_1(x)a, R_2(y)a$
$W_2(y)b, W_1(x)a, W_1(y)a, W_2(x)b$	$R_1(x)a, R_2(y)a$

3

- 线性一致性要求操作具有实时性（原子性），即操作的效果看起来是瞬间发生的。
- 顺序一致性只要求操作的最终顺序，不要求实时性。

线性一致性 = 顺序一致性 + 实时性约束。要求每个操作在调用与返回之间的某个“瞬间”原子生效，且**跨进程的操作顺序必须与真实时间先后严格一致**。

顺序一致性只要求：所有节点看到**相同的全局操作总序**，该顺序不违反**单个进程内**的程序执行顺序。**不要求符合跨进程的实时时间先后**。

1.3 因果一致性

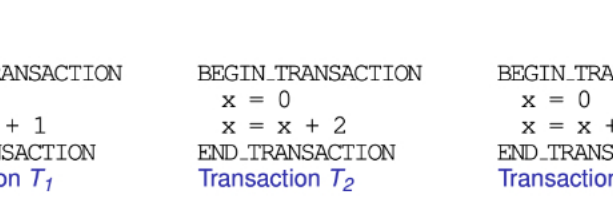
因果一致性

因果一致性 (Causal Consistency) 定义：

- 潜在因果相关的写操作必被所有进程以相同顺序看到。
- 并发的写操作可能被不同进程以不同顺序看到。



(a) 因不一致的数据存储（因为 P_2 的 $R(x)a$ 可能对 $W(x)b$ 产生影响，所以 $W(x)a \rightarrow W(x)b$ ）



(b) 因果一致的数据存储

可串行化、事务

可串行化 (Serializability)：对一组事务中的所有操作进行排序，使得最终结果与这些事务的某种串行执行相匹配。

示例：

1.4 最终一致性

最终一致性

- 最终一致性 (Eventual Consistency) 定义：**

4

BEGIN_TRANSACTION	BEGIN_TRANSACTION	BEGIN_TRANSACTION
$x = 0$	$x = 0$	$x = 0$
$x = x + 1$	$x = x + 2$	$x = x + 3$
END_TRANSACTION	END_TRANSACTION	END_TRANSACTION
Transaction T_1	Transaction T_2	Transaction T_3

A number of schedules

	Time $\rightarrow$	
S1	$x = 0 \quad x = x + 1 \quad x = 0 \quad x = x + 2 \quad x = 0 \quad x = x + 3$	Legal
S2	$x = 0 \quad x = 0 \quad x = x + 1 \quad x = x + 2 \quad x = 0 \quad x = x + 3$	Legal
S3	$x = 0 \quad x = 0 \quad x = x + 1 \quad x = 0 \quad x = x + 2 \quad x = x + 3$	Illegal
S4	$x = 0 \quad x = 0 \quad x = x + 3 \quad x = 0 \quad x = x + 1 \quad x = x + 2$	Illegal

- 考虑数据存储集合和并发写操作。在某一时刻停止更新后，所有更新都会传播，使得副本存储相同数据（直到再次接受更新）。

分布式系统一致性模型对比

一致性模型	核心特征	直观例子	实时性约束
线性一致性	操作原子生效，跨进程严格符合真实时间。	银行转账：ATM 取款后，全球查询余额立刻同步。	极强 (Real-time)
顺序一致性	所有节点看到相同的全局总序，且不违反进程内部序。	网络游戏：所有玩家看到的击杀顺序必须一致，虽然可能有延迟。	中 (Logical-time)
因果一致性	仅保证有因果关系的写操作在所有节点顺序一致。	论坛发帖：先看到主贴（因），才能看到回复（果）。	弱 (Causal-order)
最终一致性	不保证中间读取结果，但系统最终会收敛到一致。	DNS：修改记录后，全球各处生效时均不一。	无 (No guarantee)

2 客户端一致性模型

客户端一致性

核心思想：不追求全局一致，只保证单个客户端的操作历史自治。

假设通过客户端访问一个分布式数据库，你希望在 A 出更新/或读取的条目，在 B 出应该保持你在 A 出离开时的状态，即数据库对你而言是一致的。具体包括以下内涵：

- 单调读：**读不会“时光倒流”（应当越读越新）
- 单调写：**写不会“顺序颠倒”（应当先写先到）
- 读你所写：**自己写的，自己立刻能读到
- 读后写：**基于读的写，存在与读之后更新读版本（因果）

客户端一致性实现

基本思想：

- 每个写操作 W 由其源服务器分配全局唯一标识符。

5

- 为每个客户端跟踪两个写操作集：读集和写集。

实现：

- 单调读：**客户端传递读集，服务器拉取更新并更新读集。
- 单调写：**客户端传递写集，服务器拉取更新、排序并执行写操作，更新写集。
- 读你所写：**客户端传递写集，服务器拉取更新并执行读操作，更新读集。
- 读后写：**客户端传递读集，服务器拉取更新、排序并执行写操作，更新写集。

3 副本管理（对应 7.4 Replica management）

副本放置 (Replica Placement)

目标：从 N 个可能位置中找出最佳的 K 个位置。

策略：

- 选择平均客户端距离最小的位置。（计算量大）
- 选择第 K 大自治系统，并在连接最好的主机上放置服务器。（计算量大）
- 在反映延迟的 d 维几何空间中定位节点，识别密度最高的 K 个区域并在每个区域放置服务器。（计算量小）

内容复制 (Content Replication)

副本类型：

- 永久副本 (Permanent replicas)：**始终拥有副本的进程/机器，可被视为构成分布式数据存储的初始副本集。
- 服务器发起副本 (Server-initiated replica)：**是数据存储的副本，其存在是为了提升性能，并且是由数据存储的所有者主动创建的。
- 客户端发起副本 (Client-initiated replica)：**根据客户端请求动态托管副本的进程（客户端缓存）。本质上，缓存是一种本地存储设施，客户端使用它来临时存储刚刚请求的数据副本。

6

内容分发 (Content Distribution)

方法：

- 传播更新通知/失效（常用于缓存）。
- 在副本间传输数据（分布式数据库）：被动复制。
- 传播更新操作到其他副本：主动复制。

推送 (Push) 与拉取 (Pull)：

- 推送：服务器发起，无论目标是否请求。
- 拉取：客户端发起，客户端请求更新。

租约 (Leases)：

- 租约是一种契约，服务器承诺在租约到期前将更新推送给客户端。
- 自适应租约到期时间：基于年龄、续订频率、服务器状态。

管理复制对象

目标：

- 防止对同一对象的多次调用并发执行。
- 确保复制状态的所有更改都相同。

解决方案：

- 使用本地锁定机制序列化对象内部数据访问。
- 需要确定性线程调度以确保状态更改一致。

汇报完毕，欢迎同学们批评指教！
本次课大致对应课程教材第七章

7